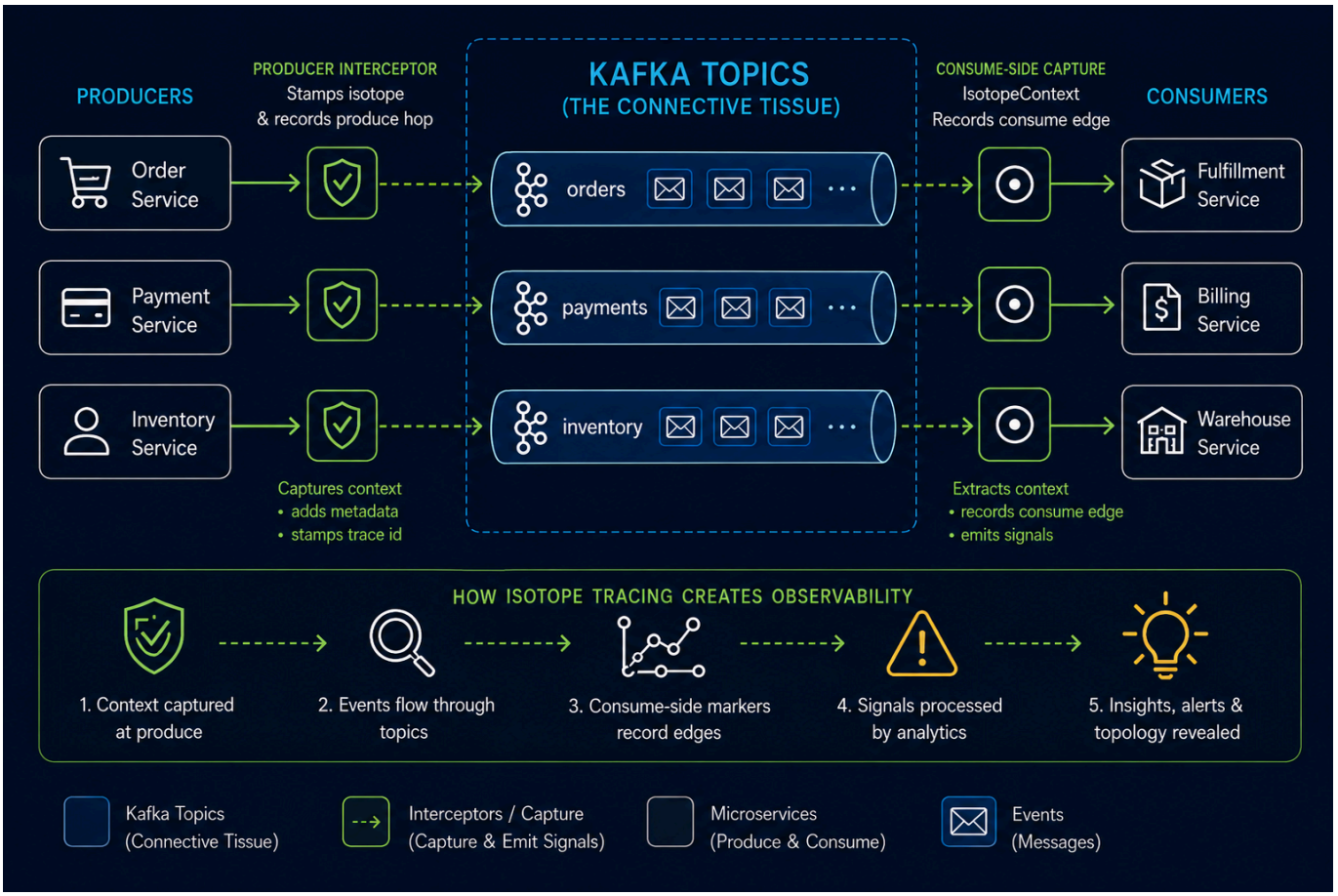
Confluent Kafka Isotope

Table of Contents

Purpose

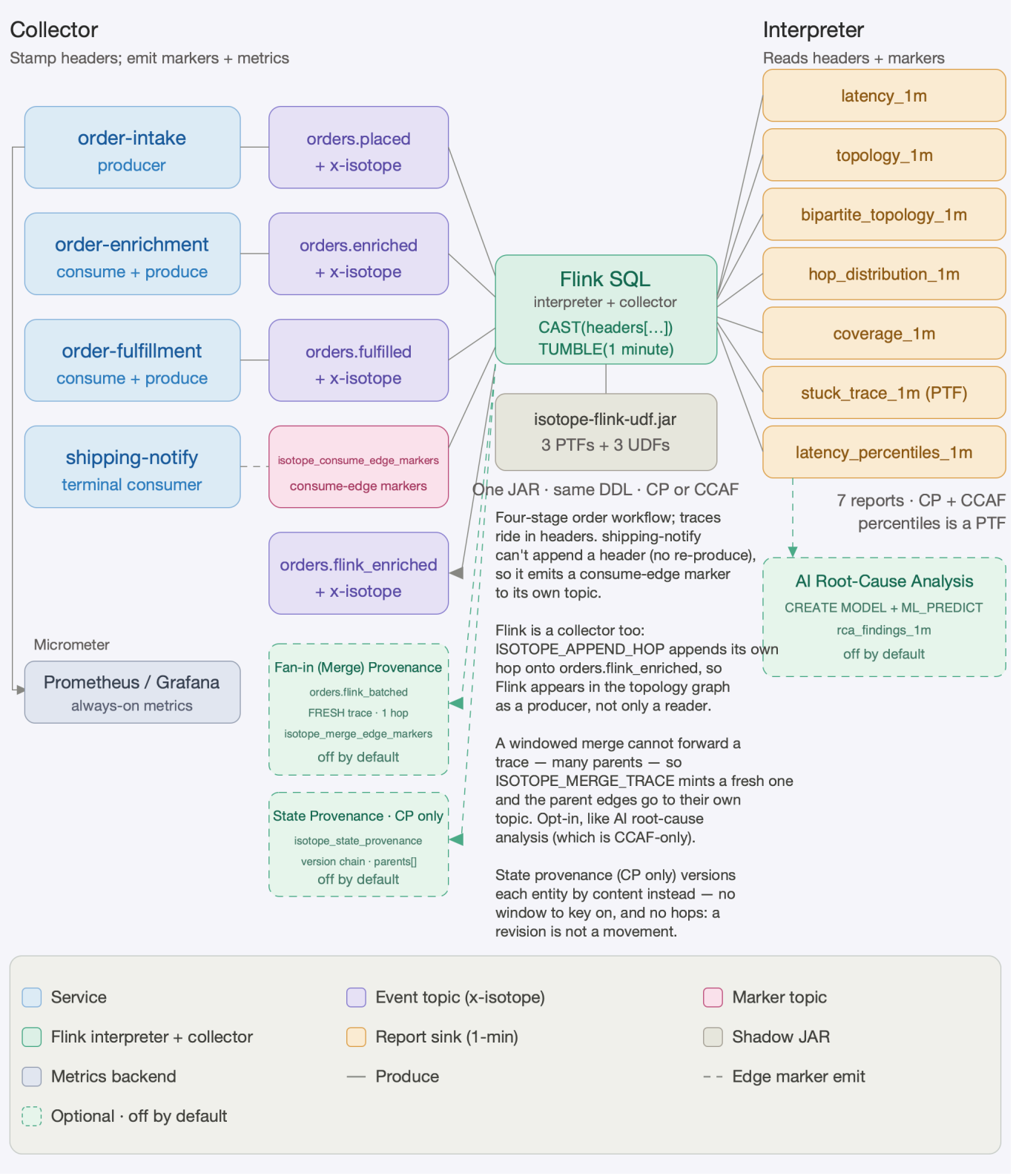
`confluent-kafka-isotope` is a reference implementation of an ***e-commerce order pipeline that uses Kafka Interceptors to capture end-to-end event-tracing data, with Prometheus and Apache Flink analyzing and reporting that data in batch and near real time***, as visualized below.



Much like isotopes used to trace molecules through a biochemical pathway, each event carries lightweight metadata that allows it to be followed as it travels through Kafka topics and distributed microservices.

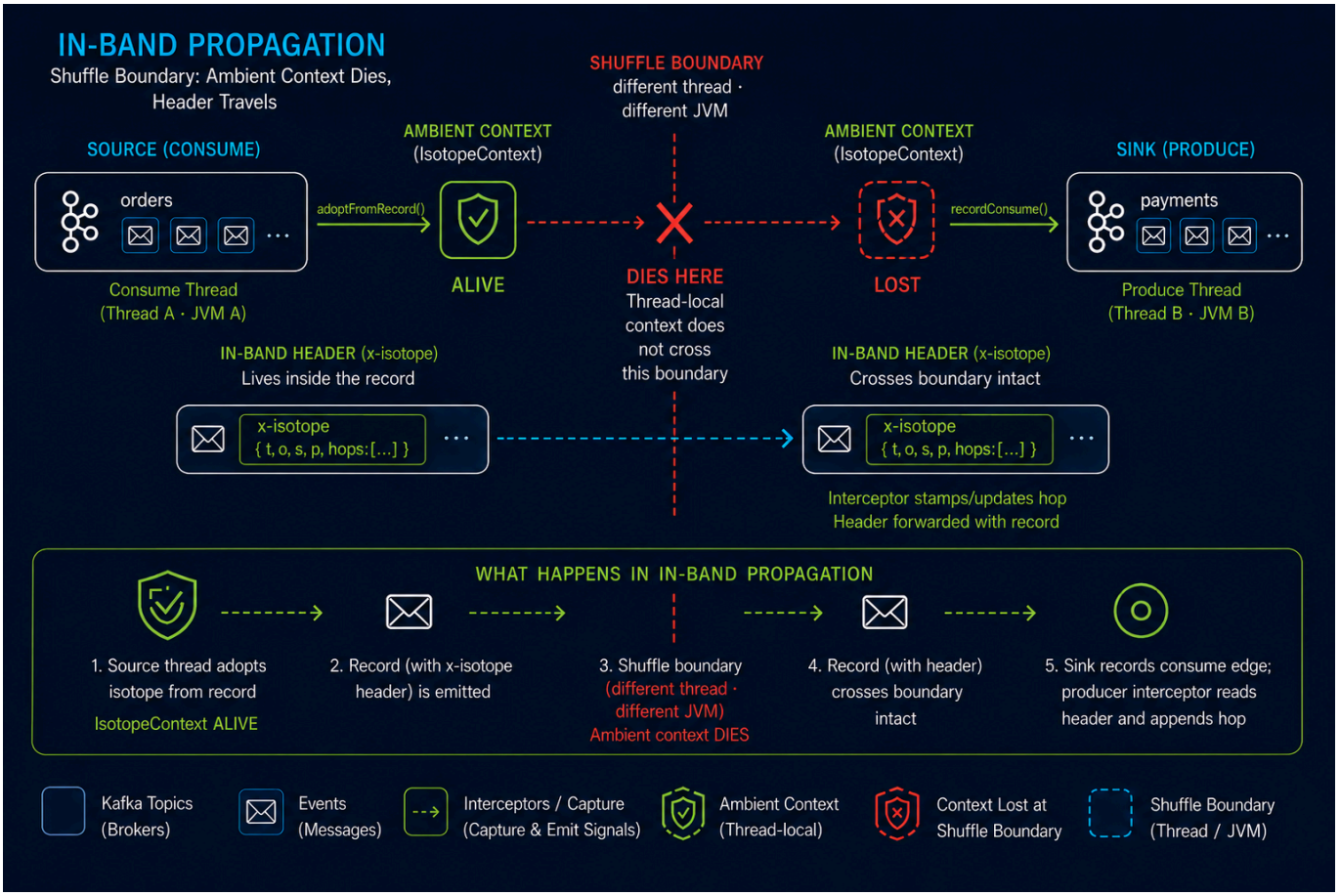
Kafka topics become the **connective tissue between services**, while **Kafka Interceptors quietly transform** the event pipeline itself into an **observable distributed system**.

This project demonstrates how **Kafka Interceptors become collectors** — inserting the isotope into record headers in place and, at terminal consumers, emitting consume-edge markers — while **Flink SQL serves as the interpreter**, reading those headers directly to produce seven 1-minute reports from a single JAR on both Confluent Platform and Confluent Cloud for Apache Flink (CCAF). Optionally, the producer interceptor can **emit Micrometer metrics to Prometheus as an always-on aggregate layer** alongside the per-trace Flink reports. (As depicted in the diagram below.)



Flink is now a collector too. A second propagation model lets a Flink statement stamp its own hop, so Flink appears in the topology graph as a producer rather than only as a reader. The interceptor propagates *out-of-band* — the in-flight isotope lives in an `IsotopeContext ThreadLocal` that `onSend()` reads on the same thread — which is correct in the services, where one thread owns a whole consume→produce hop. That cannot work in Flink: a shuffle resumes a record on a different thread in a different JVM, and Flink SQL has no user-visible thread to attach to at all. So Flink propagates *in-band*, carrying the isotope in the record's own headers via a writable `headers` metadata column and the `ISOTOPE_APPEND_HOP` scalar UDF. Two models, one wire format — every header the UDF writes is byte-

identical to the interceptor's, so the reports cannot tell the two collectors apart. See docs/flink-collector.md.



A third collector records state, not messages. The first two both record *message* lineage, and an isotope is an itinerary — one identity and its ordered hops — which is a truthful derivation record only while every step is 1:1. A row in an upsert table has no itinerary: its current value is the fold of every change that ever touched it, a DAG over versions rather than a path over messages. Asking whether a `-U`/`+U` pair is one hop or two has no good answer, because the question is wrong. So the third collector stamps **no hops at all**. `STATE_PROVENANCE` publishes one record per emitted state, identified by its *content* rather than by a window, carrying the versions it was derived from inline — which is what lets a lineage stream describing an *updating* table stay append-only end to end. Opt-in and CP-only; see docs/state-provenance.md.

That makes three collectors, distinguished by what they can truthfully record:

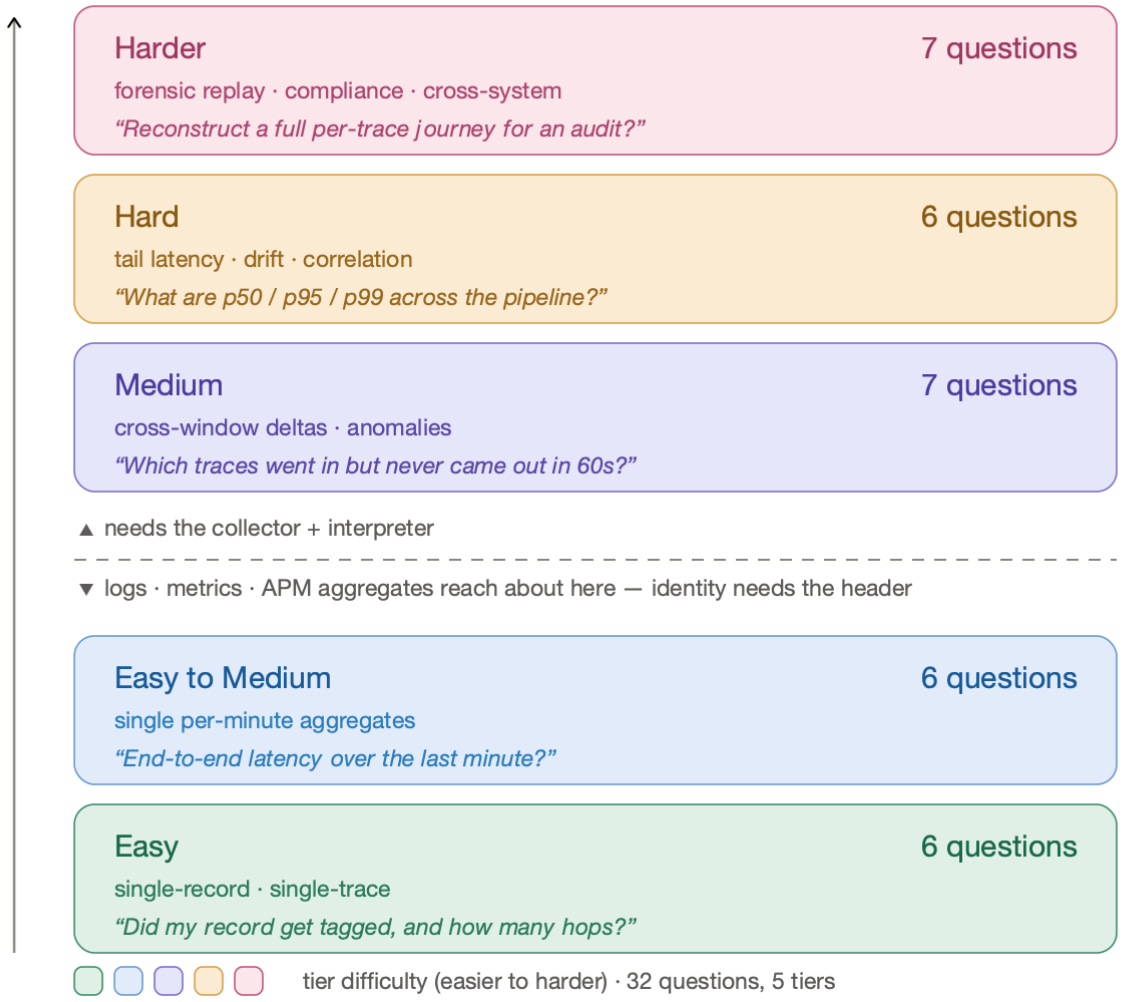
Collector	Records	Propagation	Availability
`IsotopeProducerInterceptor`	message lineage — one hop per `send()`	out-of-band (`ThreadLocal`)	always on

Collector	Records	Propagation	Availability
`ISOTOPE_APPEND_HOP`	message lineage — Flink's own hop, 1:1 statements only	in-band (record headers)	always on; opt-in fan-in variant for windowed merges mints a fresh trace rather than forwarding one (§3.5)
`STATE_PROVENANCE`	state lineage — a content-addressed version chain per entity	neither: a parallel record keyed by version	opt-in, CP only (§3.6)

The first two share one wire format, so no report can tell them apart. The third deliberately does not participate in it — which is the point: overloading the hop list to mean "revision" would fabricate movement that never happened.

With this approach, developers gain **end-to-end observability** into the flow of events through the Kafka-based microservices architecture, enabling both **real-time monitoring** and **post-hoc analysis** of event traces.

This end-to-end observability of the isotope tracing pipeline creates a **ladder of insights**, allowing developers to trace each event’s journey, identify bottlenecks, and improve the performance and reliability of the entire system. The ladder is organized into **five tiers that answer 32 questions**, as visualized below. All 32 are enumerated, tier by tier, in docs/metrics.md §6.



- ▶ Easy — single-record, single-trace
- ▶ Easy to Medium — single per-minute aggregates
- ▶ Medium — cross-window deltas, anomalies, multi-report joins
- ▶ Hard — tail behavior, drift detection, correlated analysis
- ▶ Harder — forensic replay, compliance, cross-system joins

1.0 How an Isotope Traverses an Event Pipeline

An **Isotope** is a ***lightweight tracing artifact attached to Kafka record headers***. Like a biochemical isotope used to trace molecules through a metabolic pathway, it allows the journey of a record through an event-driven architecture to be observed and analyzed.

This project models a Kafka pipeline as a **bipartite graph**: **services occupy one vertex set**, **topics the other**, and **every produce and consume operation forms an edge between them**. The resulting graph provides a unified view of the complete event topology, from producers to intermediate processing services to terminal consumers.

A `ProducerInterceptor` stamps the isotope onto each record and appends one hop for every `send()` operation, capturing the **produce edges**. Consumers call `IsotopeContext.recordConsume()` to emit a lightweight marker representing the corresponding **consume edges**.

For services that consume and then produce, Kafka Isotope supports **two complementary propagation models** for carrying the trace identity forward:

- **Out-of-band propagation** — `IsotopeContext.adoptFromRecord()` places the inbound isotope into thread-local context, where the subsequent `send()` retrieves it. This model is designed for conventional Kafka applications where the consume → produce hop remains within the same execution context.
- **In-band propagation** — the isotope remains attached to the record as it moves through the processing graph. Because propagation does not depend on thread-local state, the trace identity can survive thread, task, shuffle, network, and JVM boundaries such as those introduced by Apache Flink.

Both models produce the same Kafka isotope headers and preserve the same trace identity across hops; they differ only in **how that identity is propagated between consume and produce**: out-of-band through thread-local context, or in-band with the record itself.

Apache Flink combines the isotope headers and consume-edge markers to reconstruct the complete **service → topic → service** graph and produce seven reports:

- **End-to-end latency**
- **Latency percentiles**
- **Produce-side topology**
- **Full bipartite topology**
- **Hop distribution**
- **Per-topic coverage** — a trace-loss funnel signal
- **Stuck-trace detection**

The same implementation runs on both **Confluent Platform (self-managed)** with **Confluent Manager for Apache Flink (CMF)** and **Confluent Cloud for Apache Flink (CCAF)**.

For more in-depth discussion of the **Isotope Tracing Design**, including the two propagation models, see docs/design.md.

1.1 Anatomy of the Isotope Tracing Artifact

The isotope is a **JSON object** that travels in the `x-isotope` Kafka record header, accompanied by **seven scalar headers** that Flink SQL reads directly. The JSON carries the trace identity, origin metadata, and full ordered hop history, while the scalar headers provide a flattened view of the most-recent-hop data for easier SQL access.

- **Header `x-isotope`** (JSON bytes) carries the full trace state and hop history, forwarded by every hop:
  - `t` — 16-byte **UUIDv7** trace ID (RFC 9562): 48-bit millisecond timestamp in the high bits + 74 random bits. It remains stable for the life of the trace, and lexicographic byte order matches creation order — sorting trace IDs therefore produces chronological creation order. The random bits come from `ThreadLocalRandom`, not `SecureRandom`: a trace ID is a public observability identifier carried in Kafka headers, so the requirement is collision avoidance

rather than unpredictability, without imposing `SecureRandom` overhead on every produced record.

  - `o` — origin timestamp (ms), identical to the timestamp embedded in the UUIDv7 trace ID; retained as a separate field for typed access from Flink SQL without decoding the UUID bytes.
  - `s` — origin service name, stamped once and forwarded unchanged for the life of the trace.
  - `p` — origin pipeline name (for example, `orders` vs. `location`); like `s`, stamped once at the origin and forwarded unchanged on every hop, allowing reports to slice traces by logical pipeline.
  - `h` — ordered list of hops, each represented as `{s: service, t: topic, m: tsMs}`.
  - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted.

- **Seven scalar headers** (UTF-8 strings) carry the most-recent-hop view, allowing Flink SQL to read them directly via `CAST(headers['x-isotope-…'] AS STRING)` without parsing the JSON hop array or requiring a UDF on either CCAF or CP Flink. See scripts/flink/README.md for the complete scalar-header table.

▶ Sample Isotope Tracing Artifact

2.0 Architecture

A bird's-eye view of the moving parts. The demo CLI in `app/` consumes the external tracing library (`ai.signalroom:kafka-isotope-core`), which registers a Kafka `ProducerInterceptor` that stamps an isotope into record headers on every `send()`. Consume-then-produce services propagate the inbound trace by explicitly calling `IsotopeContext.adoptFromRecord(record)`. Business events then flow through a three-topic Kafka pipeline, where Flink SQL reads the isotope metadata and emits one-minute aggregate reports.

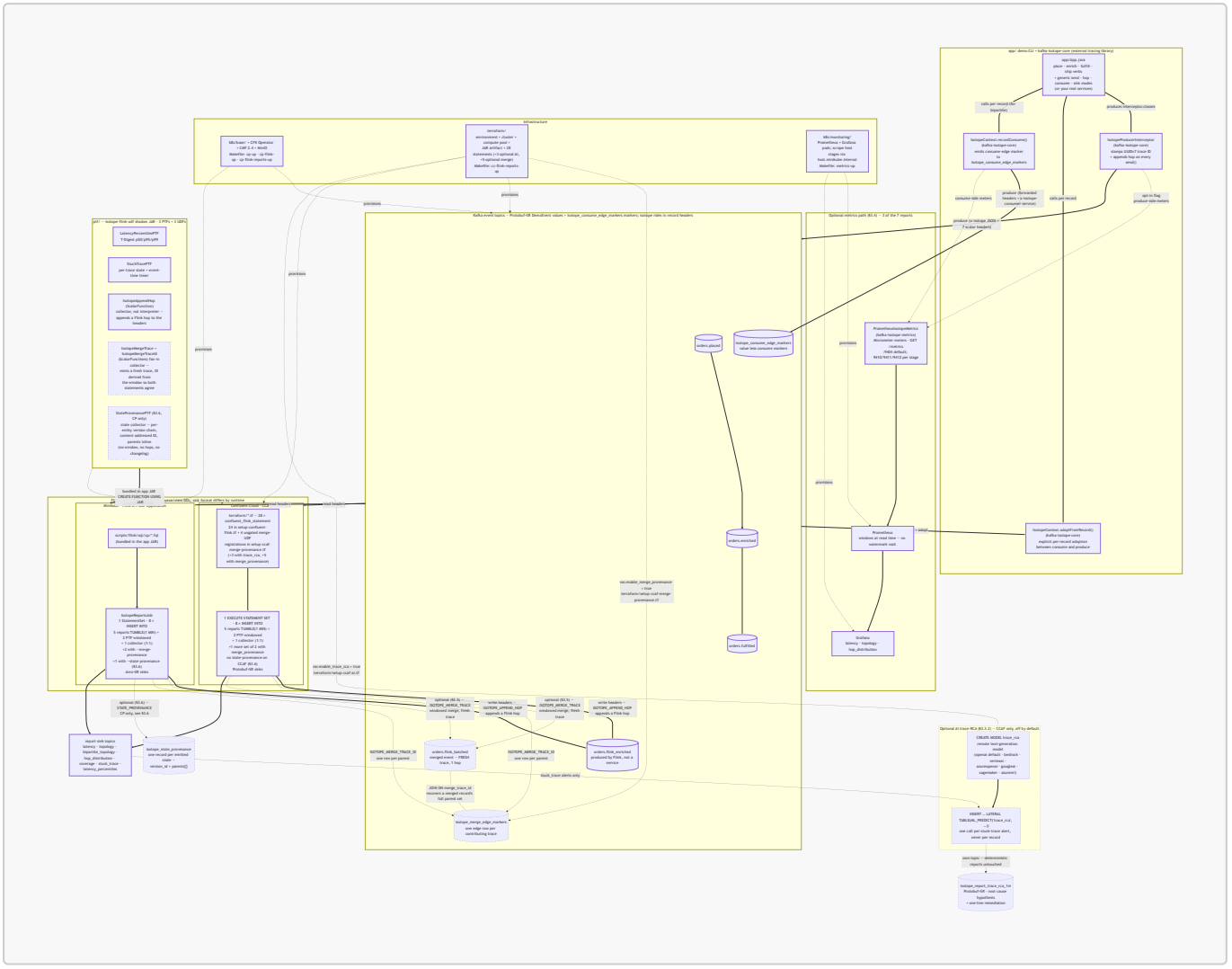
Both runtimes run the same seven logical reports off the same source and view definitions, though each runtime keeps its own copy of the SQL. **Confluent Platform (CP) + Flink** on Minikube executes the `.fql` files under `scripts/flink/sql/cp/` as a single Flink 2.1 Confluent Manager for Apache Flink (CMF) Application (`IsotopeReportsJob`), while **Confluent Cloud for Apache Flink (CCAF)** applies the same logical SQL as inline `confluent_flink_statement` Terraform resources under `terraform/`. The shadow JAR from `ptf/`—which powers two of the seven reports plus the collector UDF—runs unchanged on both runtimes: bundled into the CP application JAR and uploaded as a Flink artifact on CCAF.

Alongside the seven reports, both runtimes run one **collector** INSERT that forwards `orders.placed` to `orders.flink_enriched`, appending a Flink hop on the way. Each side runs all eight as a **single job**: CP through `IsotopeReportsJob`'s `StatementSet`, CCAF through `EXECUTE STATEMENT SET BEGIN ... END`. That matters for cost as much as symmetry — every separate CCAF statement carries its own 1-CFU floor, so eight statements meant eight floors.

Alongside that—**additive, opt-in, and disabled by default**—the interceptor can also emit **Micrometer** metrics for **Prometheus**, with **Grafana** providing visualization (§3.4 Prometheus Metrics Reporting with Grafana Visualization). This path produces the three **stateless** reports (`latency_1m`, `topology_1m`, and

`hop_distribution_1m`) without requiring a stream processor. The remaining four reports continue to run in Flink because they depend on per-`trace_id` state or absence-of-event analysis, which falls outside Prometheus's query model.

(Kafka is drawn once below for brevity; each runtime provisions its own Kafka cluster.)



▶ See Repo Layout

3.0 Getting Started

First, clone the repo to your local machine using the GitHub CLI:

```
gh repo clone j3-signalroom/confluent-kafka-isotope
```

Change to the repo directory:

```
cd /path/to/confluent-kafka-isotope
```

Then decide how you want to run the repo:

3.1 Integration Tests with Confluent Platform on Minikube

```
make install-prereqs     # docker, kubectl, minikube, helm, gettext,
gradle, openjdk17
make check-prereqs       # verify they're on PATH
```

Default Minikube sizing (override via env): `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480`, `MINIKUBE_DISK=50g`.

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start                  # one-time
make cp-up                           # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up                     # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest                                    #
every IT below
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT'   #
just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
    -PkafkaBootstrap=localhost:30092 \
    -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
`BrokerSmokeIT`	AdminClient can create/list/delete a topic via the NodePort port-forward
`ProducerInterceptorIT`	A consumer sees the `x-isotope` JSON header + all 7 scalar reporting headers with the expected origin/hop values, and the

Test	What it verifies
	Protobuf round-trip preserves `DemoEvent.source` / `payload`
`ThreeStageHopPropagationIT`	`order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service` produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = `order-intake-service`, this = `order-enrichment-service`, hop count = 2) at the terminal; consume-then-produce hops use `IsotopeContext.adoptFromRecord` to carry the trace forward
`BipartiteTopologyIT`	The 4-stage `order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service → topic-CD → shipping-notification-service` chain emits exactly three consume-edge markers to a per-test markers topic — one per consume edge. Every marker carries the trace ID, forwarded `x-isotope-*` scalars describing the upstream producer, and the new `x-isotope-consumer-service` naming the downstream consumer. Asserts the `(consumer_service, consumed_topic)` set is exactly the three pairs of stages 2-4

3.2 Seven Scalar Headers Flink SQL Reports with Apache Flink on Minikube

> **Caveat:** Minikube is a single-node cluster. It is not a production-like environment, but it is sufficient for local development and testing. The Confluent Platform (CP) + Flink stack runs in Minikube, and you can port-forward Kafka and Schema Registry to your local machine.

To ***run***, ***test***, and ***debug*** Apache Flink like a production engineer, this project provides a full Confluent Platform + Flink stack running locally on Minikube — no cloud required.

You get an environment on your machine, with all the components you'd expect in a real deployment:

- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.2** via the Confluent Flink Kubernetes Operator 1.140.1
- **Confluent Manager for Apache Flink (CMF) 2.4.0** for Flink environment management

To run this project, you'll need **macOS (with Homebrew)** or **Linux (with apt-get)**. The full stack — **Minikube + Confluent Platform + Flink + CMF** — is resource-intensive and designed to mirror an adequate development environment. Therefore, the following defaults are recommended:

Resource	Default
CPUs	6
Memory	20 GB
Disk	50 GB

> These settings ensure stable performance across all components. You can tune them as needed, but lower resource levels may cause pod restarts or degraded performance.

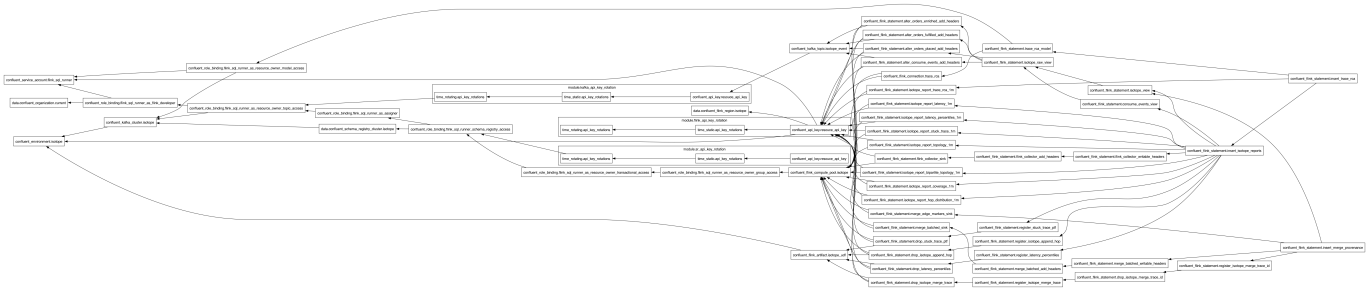
Seven reports — five pure Flink SQL plus two JAR-backed PTFs — and the collector INSERT run as a **single Flink 2.1 CMF Application** (`IsotopeReportsJob`, one StatementSet, eight sinks) submitted through CMF and executed by the Confluent Flink Kubernetes Operator. One StatementSet means one failure domain: a fault in any INSERT stops them all, so a missing sink topic takes the reports down with it (which is why `deploy-cmf-flink-reports.sh` pre-creates every sink — unlike CCAF, OSS Flink's Kafka connector declares a table over a topic that must already exist rather than creating it). No raw session cluster is involved; `k8s/base/flink-cluster-deployment.yaml` (`make flink-deploy` / `make flink-sql`) is a separate, optional path for ad-hoc SQL. Confluent Cloud runs the same seven reports from its own copy of the SQL, inlined in Terraform — see §3.3 Confluent Cloud for Apache Flink for that path; this section is the local-Minikube one.

The full bring-up sequence — cluster → Flink → reports → traffic → teardown — is consolidated in docs/runbook-minikube.md. The short version: `make cp-flink-up` then `make cp-flink-reports-up`, then drive traffic across **multiple** 1-minute windows (a single burst sits in one open window forever — the watermark has to cross `window_end` for a tumbling window to emit) and wait ~90s after the last record.

Report sink topics ride **Avro+SR** (`avro-confluent`, auto-registered on first write) so Control Center renders them natively — a deliberate *format-by-domain* split: app events are **Protobuf+SR** (`DemoEvent`), Flink aggregates are **Avro+SR** (cp-flink ships no SR-integrated Protobuf format), and the consume-edge marker topic `isotope_consume_edge_markers` is **null-value / headers-only**. Not a defect — a clean split by domain.

3.3 Seven Scalar Headers Flink SQL Reports with Confluent Cloud for Apache Flink

The Confluent Cloud for Apache Flink (CCAF) parallel of §3.2 CP + Apache Flink on MiniKube, driven by Terraform under terraform/. The Terraform graph below depicts the resources deployed in Confluent Cloud.



`make cc-flink-reports-up CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...` provisions a fresh Confluent Cloud environment containing:

- Kafka cluster
- Compute pool
- Two rotating service-account API key pairs (Kafka + Schema Registry)
- The uploaded PTF JAR
- 24 `confluent_flink_statement` resources (6 `ALTER TABLE` + 3 `VIEW` + 8 sink `CREATE TABLE` + 3 `CREATE FUNCTION`, plus 1 `EXECUTE STATEMENT SET` holding all 8 `INSERT INTO`s and 3

transient `DROP FUNCTION`)

The full provision → deploy → traffic → teardown sequence is consolidated in docs/runbook-ccaf.md. `make cc-flink-reports-up` (~6–8 min first run; idempotent re-applies), drive traffic with `scripts/cc-app-run.sh place|enrich|fulfill|ship` across **multiple** 1-minute windows (a single burst sits in one open window forever, and you wait ~90s after the last record), then `make cc-flink-reports-down` to `terraform destroy` the whole environment.

Format-by-runtime (not-by-domain). CP's reports land on **Avro+SR** (`'value.format' = 'avro-confluent'` in scripts/flink/sql/cp/05_report_sinks.fql). CCAF's reports land on **Protobuf+SR** (`'value.format' = 'proto-registry'` in each sink's `WITH` clause in terraform/setup-confluent-flink.tf). The two runtimes' SQL is otherwise unshared: CP's lives hardcoded in scripts/flink/sql/cp/, CCAF's lives inline as `confluent_flink_statement` resources in terraform/setup-confluent-flink.tf.

3.3.1 Why `latency_percentiles` is a `ProcessTableFunction` (PTF)

Since CCAF does not support User-Defined AGGregate functions (UDAGG), I implemented the percentiles report as a `ProcessTableFunction` to keep it portable across runtimes. `LATENCY_PERCENTILES` (class `LatencyPercentilesPTF`) performs its own 1-minute tumbling-window aggregation over a T-Digest sketch using per-window state and event-time timers. **A PTF avoids that UDAGG restriction, so it registers and runs on both runtimes, just like `STUCK_TRACE_PTF`. Both runtimes therefore run the same seven reports:** `latency` (avg/min/max), `topology` (produce-side), `bipartite_topology` (full service↔topic↔service graph), `hop_distribution`, `coverage`, `stuck_trace`, and `latency_percentiles` (p50/p95/p99).

3.3.2 [OPTIONAL] Eighth Report — AI Root-Cause Analysis (RCA)

In addition to the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis and a one-line remediation. This **CCAF-only capability** is configured by `terraform/setup-ccaf-ai.tf` and can be enabled **at deploy time** by running:

```
make cc-flink-reports-up  CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
ENABLE_TRACE_RCA=true RCA_MODEL_API_KEY=... RCA_MODEL_PROVIDER=...
RCA_MODEL_VERSION=... RCA_MODEL_ENDPOINT=...
```

The deployment provisions the standard CCAF environment used by the seven deterministic reports:

- Kafka cluster
- Compute pool
- Two rotating service-account API key pairs (Kafka + Schema Registry)
- The uploaded PTF JAR
- 24 `confluent_flink_statement` resources (6 `ALTER TABLE` + 3 `VIEW` + 8 sink `CREATE TABLE` + 3 `CREATE FUNCTION`, plus 1 `EXECUTE STATEMENT SET` holding all 8 `INSERT INTO`s and 3 transient `DROP FUNCTION`)

When `ENABLE_TRACE_RCA=true`, the deployment **additionally provisions the AI RCA resources**:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT … SELECT … LATERAL TABLE(ML_PREDICT('trace_rca', …))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

> The default provider is OpenAI (`gpt-4o`). Claude is supported via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

3.4 [OPTIONAL] Prometheus Metrics Reporting with Grafana Visualization

Three of the seven reports — `latency_1m`, `topology_1m`, and `hop_distribution_1m` — are pure **stateless scalar aggregations** over bounded-cardinality dimensions (`service` / `topic` / `hop_count`, never `trace_id`). These don't require a stream processor: the producer interceptor already has every value in scope on every `send()`, so it emits them as **Micrometer** meters (PrometheusIsotopeMetrics.java). **Prometheus** performs the windowed aggregations at query time, with **Grafana** providing the visualization layer.

The other four reports — `latency_percentiles`, `coverage`, `bipartite_topology`, and `stuck_trace` — require per-`trace_id` state or absence-of-event detection that Prometheus cannot express, so they **remain in Flink**. This path is **additive and opt-in**: a **3-Micrometer / 4-Flink split**.

> **Full details** — including the meter/PromQL reference, the produce- and consume-side signals, the two deliberate gaps (`distinct_traces`, windowed `min`), and why latency percentiles remain implemented as a PTF — are in **docs/metrics.md**. The one-command Prometheus + Grafana showcase has its own runbook: **k8s/monitoring/README.md** (`make metrics-up`).

3.5 [OPTIONAL] Fan-in (Merge) Provenance

The Flink collector is **1:1 by design** — it appends a hop to records it forwards one-for-one. That is tracing, and a trace is only a truthful derivation record while every step has exactly one parent. A windowed aggregate breaks that: a `SUM` over 1,000 records has 1,000 parents, and forwarding one of their trace IDs would not be incomplete provenance — it would **fabricate** provenance.

This optional path records the fan-in case honestly, without changing the isotope wire format. The merged record on `orders.flink_batched` carries a **fresh** trace with one hop, and the many-to-one edges go to `isotope_merge_edge_markers` — one row per contributing trace per window, weighted by `contributing_records` — the same architectural pattern `isotope_consume_edge_markers` already uses for consume edges. Join the two on `merge_trace_id` to recover any merged record's full parent set.

Both runtimes use the same switch, off by default:

```
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true

make cc-flink-reports-up ENABLE_MERGE_PROVENANCE=true \
```

```
    CONFLUENT_API_KEY=$CONFLUENT_API_KEY
CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET
```

Disabled, neither runtime creates a table, a topic, or a statement for it, and `isotope_raw`, the typed views, and all seven reports are untouched either way.

> **Full details** — why the merge trace ID must be *derived* from the window rather than minted (two statements have to agree on it independently), why it is two INSERTs rather than one PTF, and the two costs worth knowing about — are in **docs/flink-collector.md §2.4**.

3.6 [OPTIONAL] State-Level Provenance

Fan-in provenance above answers "which records produced this merged output?" — but it needs a **window** to derive the merged record's identity from. That rules out the entire updating world: an unbounded `GROUP BY`, a regular join, and a dedup all produce a changelog whose output row is revised as inputs arrive, with a different parent set each time and no window bound to hash. Point the reports at `upsert-kafka` or CDC sources and most of them will not even plan.

This optional path answers the same question for **state**: not "where did this message go" but "which versions produced the current value of this row." Identity is **content-addressed** rather than window-derived — `StateVersion` hashes `(source_name, entity_key, content)` into a UUIDv7 whose high bits carry the event time — so it needs no window at all, and a version is never revised, only superseded. That single choice is what makes a lineage stream describing an *updating* table itself **append-only**, so no operator in the pipeline ever consumes a changelog.

`STATE_PROVENANCE` is a `ProcessTableFunction` keyed by entity, publishing one record per emitted state to `isotope_state_provenance`, with the versions it came from carried **inline** in a `parents` array. That is deliberately unlike the merge collector's two-statement shape: the parent set is a column of the record it describes, so an output and its parents cannot drift apart, and no second statement has to independently re-derive an ID. A flat edge table, if wanted, is an `UNNEST` projection of this topic.

Note that this collector **does not stamp hops**. A row being revised is not a record taking a hop, and asking whether a `-U`/`+U` pair is one hop or two has no good answer because the question is wrong. Message lineage stays with the isotope; state lineage is a parallel record keyed by version. The two coexist without either lying.

```
make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true

# both collectors at once — they answer different questions
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
ENABLE_STATE_PROVENANCE=true
```

CP only, for one specific reason. The version preimage needs bytes that are stable for a given state, and CP's source tables are declared `'value.format' = 'raw'`, so the raw value is available as a `BYTES` column. CCAF's Topic Catalog imports each topic with typed Protobuf columns instead and does not hand back the raw value, so the same statement cannot be written there verbatim — it would hash a canonical rendering of the typed columns, which is a legitimate design but yields IDs that differ from CP's

for identical data. The PTF itself is portable (its state is plain `String`/`List`, which is what CCAF requires), so this is a *canonicalization* gap, not a capability gap. There is no `ENABLE_STATE_PROVENANCE` on `make cc-flink-reports-up`.

> **Full details** — the identity model, why one operator instead of two statements, the CCAF assessment, and the limits (unbounded parent sets, no recursive ancestry in Flink SQL, compaction bounding replay) — are in **docs/state-provenance.md**.

Resources

- Medium Article: Kafka's quiet observability superpower — Kafka Interceptors
- Medium Article: Kafka's quiet observability superpower — Kafka Interceptors with assistance from AI